

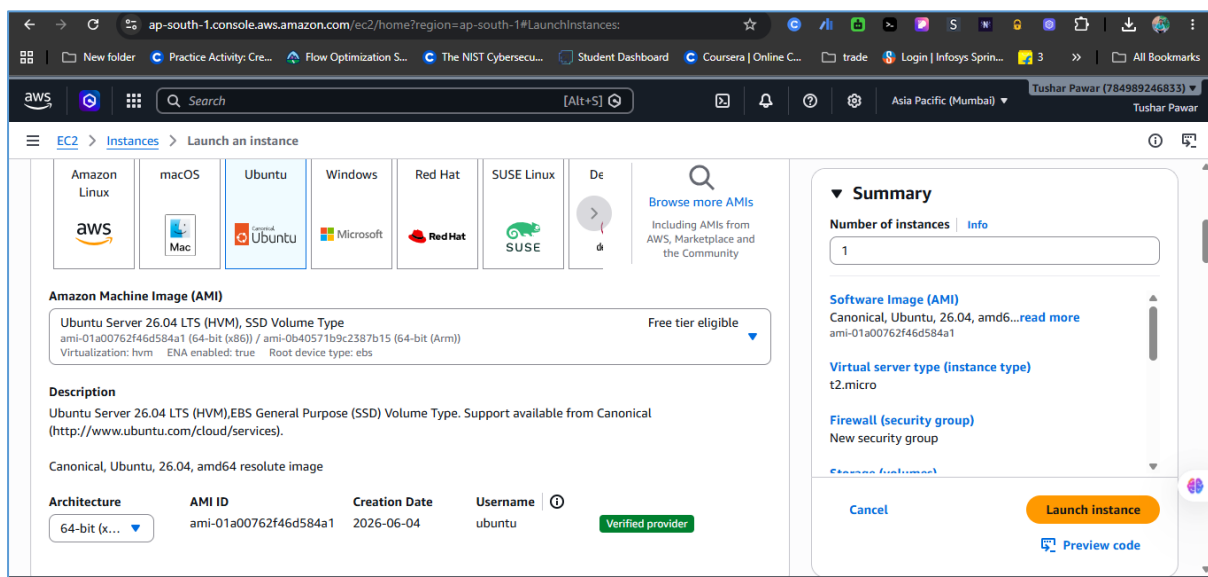


Jenkins CI/CD Pipeline

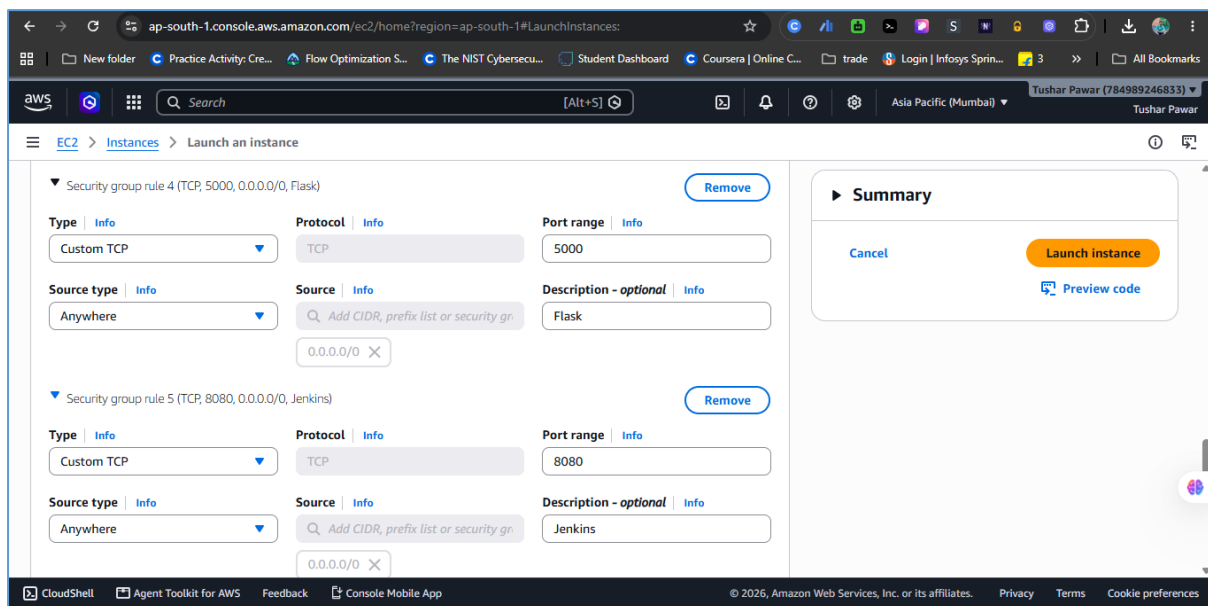
DEVOPS | Assignment

Objective: Deploy a Flask backend and an Express frontend on an Amazon EC2 instance, then implement a CI/CD pipeline using Jenkins to automate the deployment process.

1. GitHub repositories (flask-backend and express-frontend) successfully created and displayed on my GitHub profile. Both repositories are public and contain the complete source code for the applications.
2. This screenshot displays the EC2 instance details page showing the instance ID, public IPv4 address (65.1.100.135), instance type (t2.medium/t2.micro), and instance state (running). The Ubuntu 22.04 LTS AMI is confirmed in the instance configuration.



3. This screenshot shows the inbound rules configured for the Jenkins-CICD security group. All required ports are open: SSH (22), HTTP (80), Express (3000), Flask (5000), and Jenkins (8080). The rules allow access from appropriate sources to enable application accessibility.



4. This screenshot shows successful SSH connection to the EC2 instance using the jenkins-cicd-key.pem file. The terminal displays the Ubuntu welcome message and confirms the connection to ubuntu@65.1.100.135, demonstrating successful remote access to the server.
5. This screenshot captures the system update process running on the EC2 instance. The command 'sudo apt update && sudo apt upgrade -y' is shown executing, with package lists being updated and system packages being upgraded to their latest versions.

```

ubuntu@ip-172-31-10-120: ~
Scanning linux images...

Pending kernel upgrade!
Running kernel version:
7.0.0-1006-aws
Diagnostics:
  The currently running kernel version is not the expected kernel version 7.0.0-1012-aws.

Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.

Restarting services...
systemctl restart acpid.service chrony.service cron.service multipathd.service polkit.service

Service restarts being deferred:
systemctl restart ModemManager.service
/etc/needrestart/restart.d/dbus.service
systemctl restart getty@tty1.service
systemctl restart networkd-dispatcher.service
systemctl restart serial-getty@ttyS0.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
ubuntu @ session #1: apt[1504], bash[1321], sshd-session[1201,1318], sudo[1501,1503]
ubuntu @ user manager: (sd-pam)[1208]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-10-120:~$

```

6. This screenshot displays the successful installation of all required dependencies. The terminal shows version outputs for Python 3 (3.10+), pip3, Node.js (18.x), npm, PM2, Git, and Java 11, confirming that all necessary tools are properly installed and accessible.

```
ubuntu@ip-172-31-10-120: ~  
No VM guests are running outdated hypervisor (qemu) binaries on this host.  
ubuntu@ip-172-31-10-120:~$ node --version  
v18.20.8  
ubuntu@ip-172-31-10-120:~$ npm version  
{  
  npm: '10.8.2',  
  node: '18.20.8',  
  acorn: '8.14.0',  
  ada: '2.8.0',  
  ares: '1.29.0',  
  base64: '0.5.2',  
  brotli: '1.1.0',  
  cjs_modulelexer: '1.2.2',  
  cldr: '44.1',  
  icu: '74.2',  
  llhttp: '6.1.1',  
  modules: '108',  
  napi: '9',  
  nghttp2: '1.61.0',  
  openssl: '3.0.16',  
  simdutf: '5.6.4',  
  tz: '2024a',  
  undici: '5.29.0',  
  unicode: '15.1',  
  uv: '1.44.2',  
  uvwasi: '0.0.19',  
  v8: '10.2.154.26-node.39',  
  zlib: '1.3.0.1-motley'  
}
```

7. This screenshot shows the Jenkins initial admin password retrieved from `/var/lib/jenkins/secrets/initialAdminPassword`. This password is required for the first-time Jenkins setup and unlocking the Jenkins interface.

```
ubuntu@ip-172-31-10-120: ~  
ubuntu@ip-172-31-10-120:~$ sudo cat /var/lib/jenkins/secrets/initialAdminPassword  
2a05eeaaaa93478fb9f7796b544e7e61  
ubuntu@ip-172-31-10-120:~$
```

8. This screenshot shows the curl command testing the Flask health endpoint (`http://localhost:5000/api/health`). The JSON response displays `{"status": "healthy", "message": "Flask backend is running"}`, confirming that the Flask API is operational and responding correctly.

```
22 packages are looking for funding  
run `npm fund` for details  
3 moderate severity vulnerabilities  
To address all issues, run:  
npm audit fix  
Run `npm audit` for details.  
ubuntu@ip-172-31-10-120:~/apps/express-frontend$ pm2 start npm --name express-f  
rontend -- start  
[PM2] Starting /usr/bin/npm in fork_mode (1 instance)  
[PM2] Done.  


| id | name             | mode | ⌵ | status | cpu | memory |
|----|------------------|------|---|--------|-----|--------|
| 1  | express-frontend | fork | 0 | online | 0%  | 15.6mb |
| 0  | flask-backend    | fork | 0 | online | 0%  | 35.8mb |

  
host metrics | cpu: 0.1% | ram usage: 70.2%  
ubuntu@ip-172-31-10-120:~/apps/express-frontend$ pm2 status  


| id | name             | mode | ⌵ | status | cpu | memory |
|----|------------------|------|---|--------|-----|--------|
| 1  | express-frontend | fork | 0 | online | 0%  | 53.6mb |
| 0  | flask-backend    | fork | 0 | online | 0%  | 35.8mb |

  
host metrics | cpu: 2.5% | ram usage: 72%  
ubuntu@ip-172-31-10-120:~/apps/express-frontend$
```

9. This screenshot displays PM2 status with both applications running. The flask-backend and express-frontend processes are both shown with status "online", demonstrating successful deployment and process management of both applications simultaneously.

```
ubuntu@ip-172-31-10-120: ~/apps/express-frontend$ pm2 start npm --name express-f
rontend -- start
[PM2] Starting /usr/bin/npm in fork_mode (1 instance)
[PM2] Done.
```

id	name	mode	u	status	cpu	memory
1	express-frontend	fork	0	online	0%	15.6mb
0	flask-backend	fork	0	online	0%	35.8mb

```
host metrics | cpu: 0.1% | ram usage: 70.2%
ubuntu@ip-172-31-10-120:~/apps/express-frontend$ pm2 status
```

id	name	mode	u	status	cpu	memory
1	express-frontend	fork	0	online	0%	53.6mb
0	flask-backend	fork	0	online	0%	35.8mb

```
host metrics | cpu: 2.5% | ram usage: 72%
ubuntu@ip-172-31-10-120:~/apps/express-frontend$ pm2 save
[PM2] Saving current process list...
[PM2] Successfully saved in /home/ubuntu/.pm2/dump.pm2
ubuntu@ip-172-31-10-120:~/apps/express-frontend$ pm2 startup
[PM2] Init System found: systemd
[PM2] To setup the Startup Script, copy/paste the following command:
sudo env PATH=$PATH:/usr/bin /usr/lib/node_modules/pm2/bin/pm2 startup systemd
-u ubuntu --hp /home/ubuntu
ubuntu@ip-172-31-10-120:~/apps/express-frontend$
```

```
usr/games:/usr/local/games:/snap/bin:/usr/bin:/bin:/usr/local/sbin:/usr/local/b
in:/usr/sbin:/usr/bin
Environment=PM2_HOME=/home/ubuntu/.pm2
PIDFile=/home/ubuntu/.pm2/pm2.pid
Restart=on-failure

ExecStart=/usr/lib/node_modules/pm2/bin/pm2 resurrect
ExecReload=/usr/lib/node_modules/pm2/bin/pm2 reload all
ExecStop=/usr/lib/node_modules/pm2/bin/pm2 kill

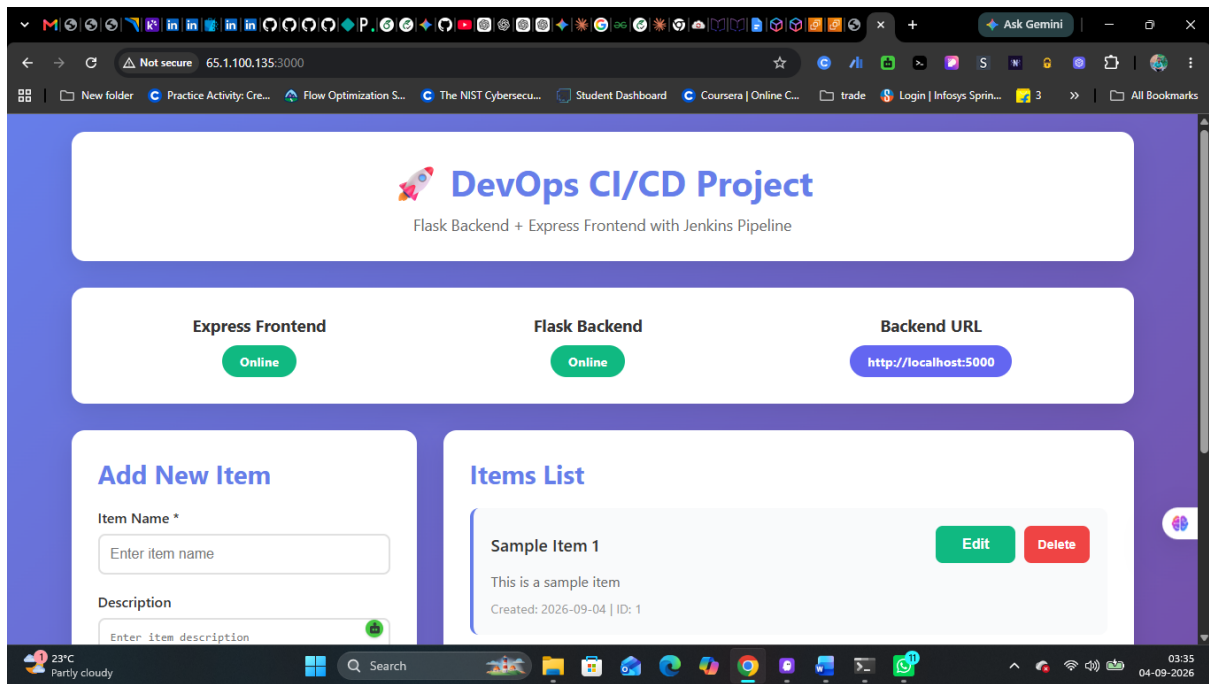
[Install]
WantedBy=multi-user.target

Target path
/etc/systemd/system/pm2-ubuntu.service
Command list
[ 'systemctl enable pm2-ubuntu' ]
[PM2] Writing init configuration in /etc/systemd/system/pm2-ubuntu.service
[PM2] Making script booting at startup...
[PM2] [-] Executing: systemctl enable pm2-ubuntu...
Created symlink '/etc/systemd/system/multi-user.target.wants/pm2-ubuntu.service'
-> '/etc/systemd/system/pm2-ubuntu.service'.
[PM2] [v] Command successfully executed.

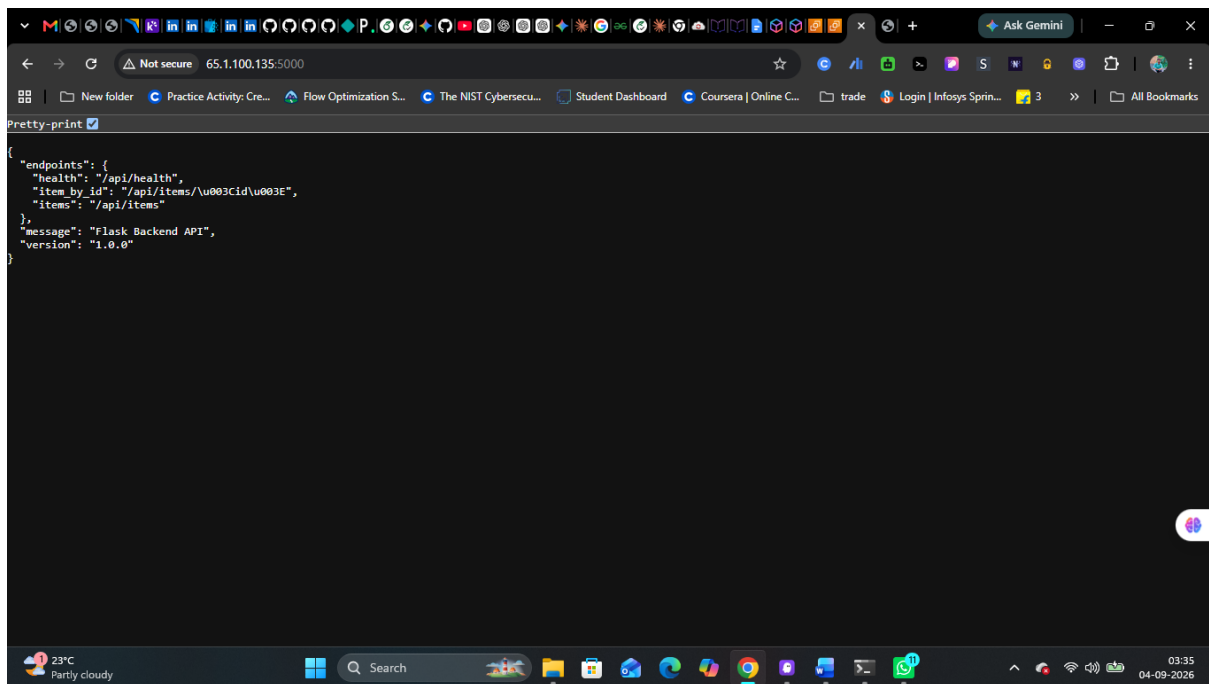
+-----+
[PM2] Freeze a process list on reboot via:
$ pm2 save

[PM2] Remove init script via:
$ pm2 unstartup systemd
ubuntu@ip-172-31-10-120:~/apps/express-frontend$ |
```

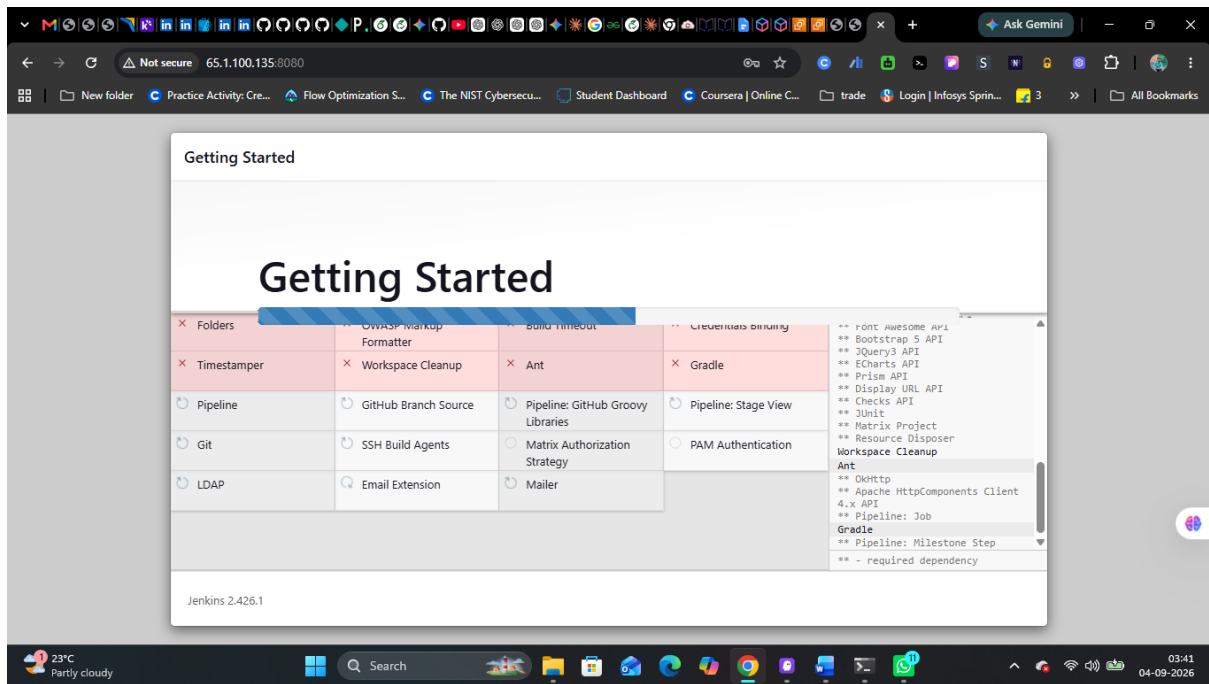
10. This screenshot shows the Express frontend web interface accessed at <http://65.1.100.135:3000>. The beautiful gradient UI is displayed with the heading "DevOps CI/CD Project", status indicators showing both frontend and backend online, and the items management interface, confirming full functionality of the web application.



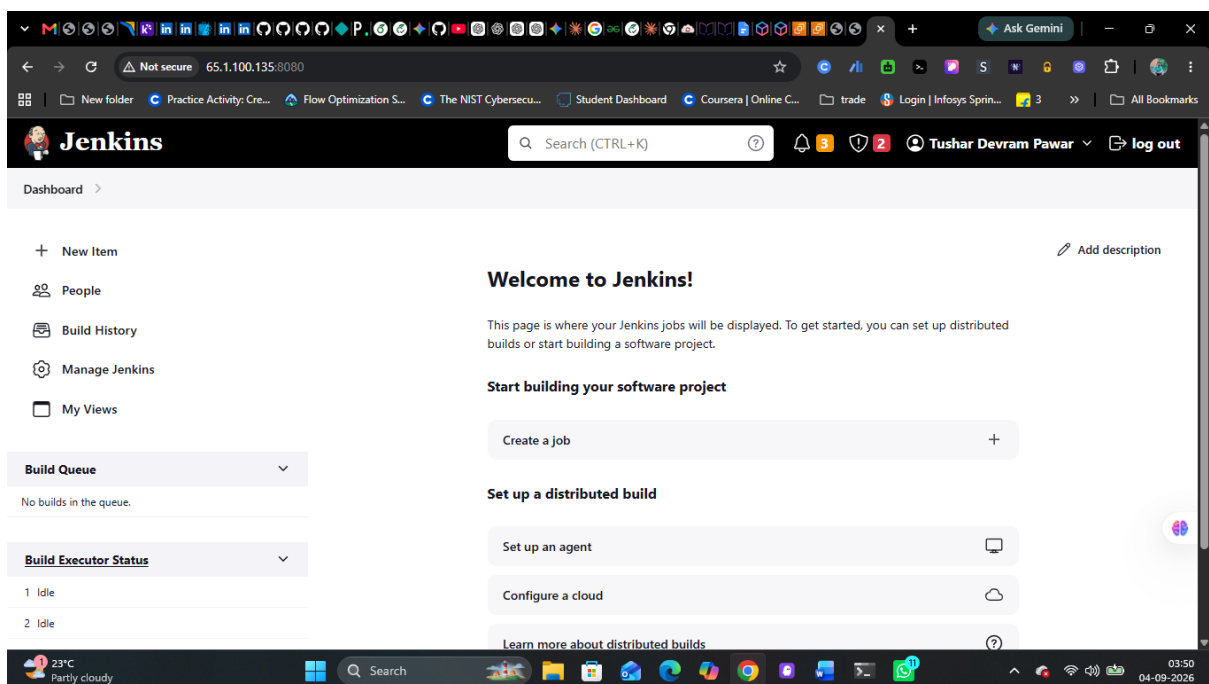
11. This screenshot shows the Flask backend API accessed through a web browser at <http://65.1.100.135:5000>. The page displays the API information including available endpoints, version, and welcome message, confirming public accessibility of the Flask backend.



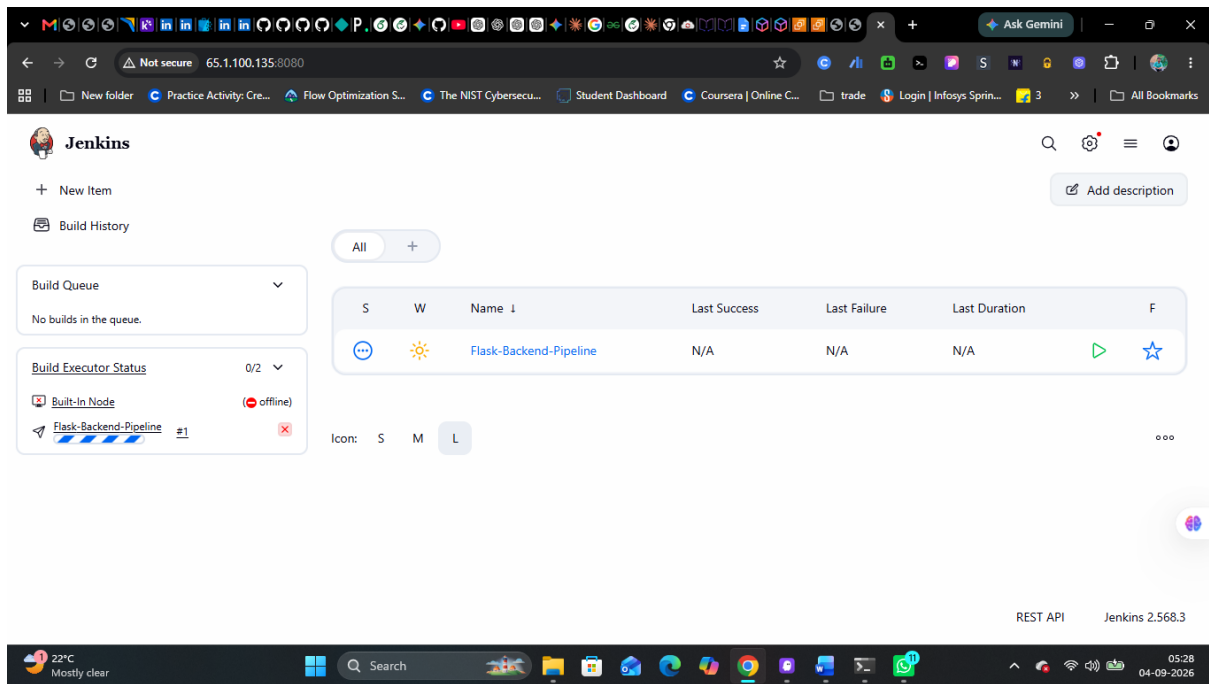
12. This screenshot captures the Jenkins plugin installation page showing the "Getting Started" wizard with suggested plugins being installed. Multiple plugins are shown downloading and installing simultaneously, including Git, Pipeline, Folders, and other essential Jenkins plugins.



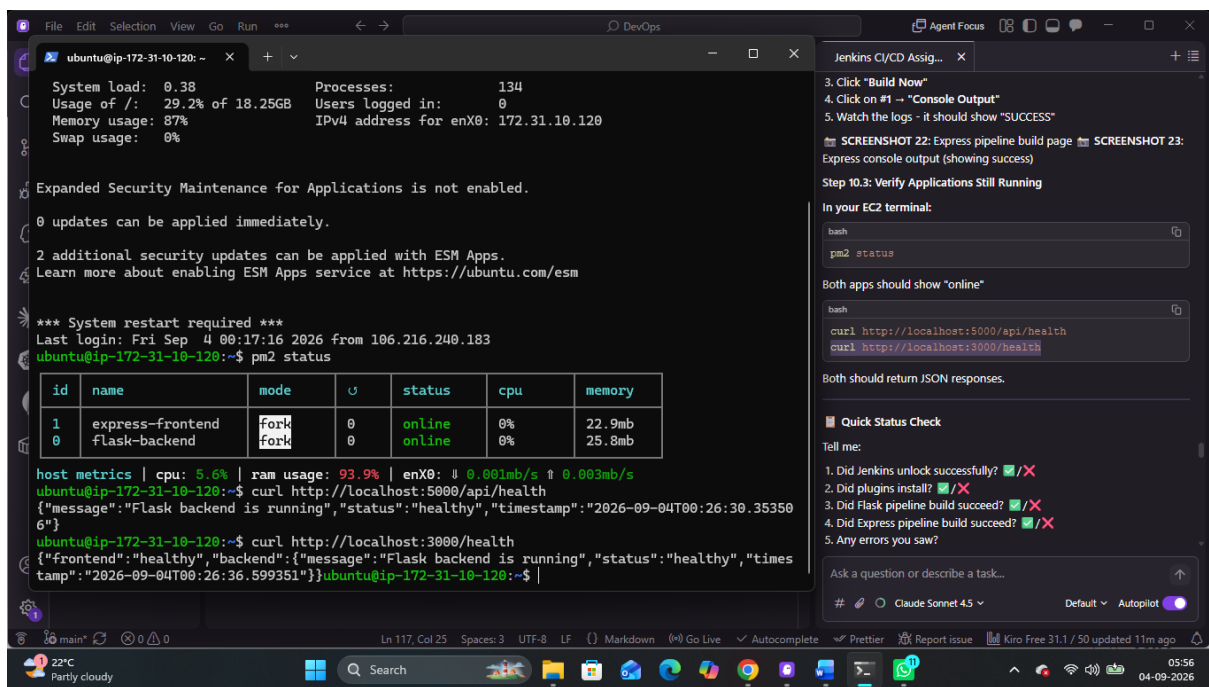
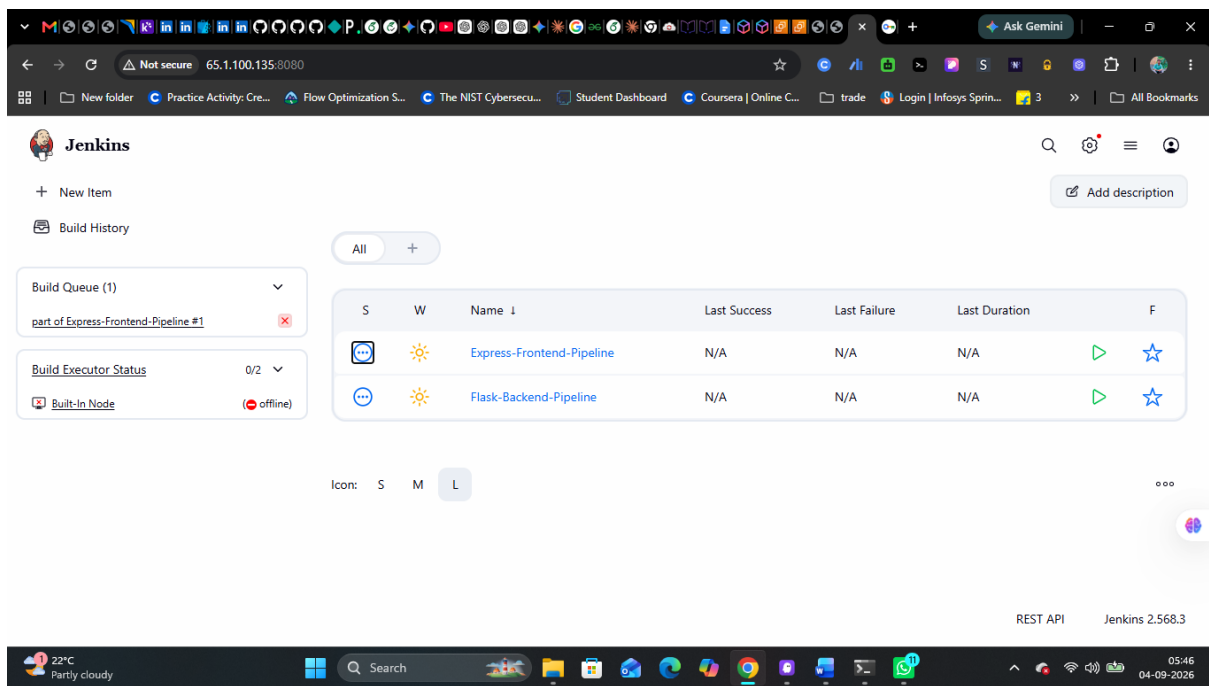
13. This screenshot displays the main Jenkins dashboard after successful setup. The interface shows the "Welcome to Jenkins!" message, navigation menu on the left, and options to create new items. Both Flask-Backend-Pipeline and Express-Frontend-Pipeline are visible in the project list.



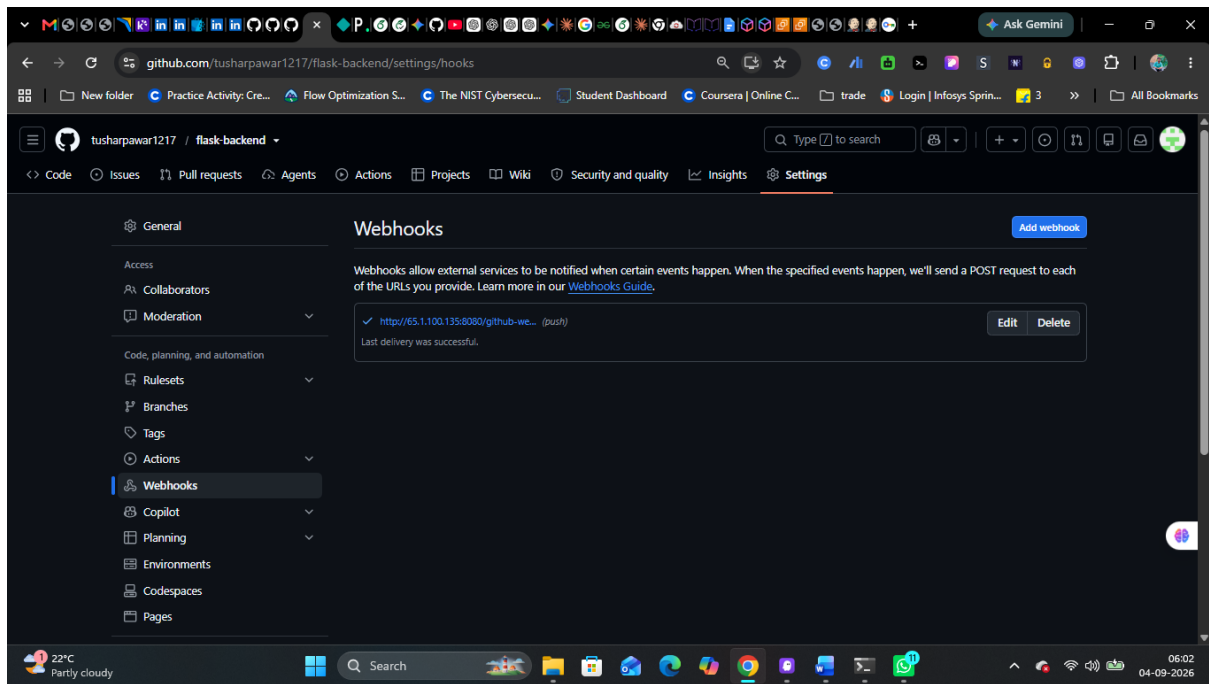
14. This screenshot shows the configuration page for Flask-Backend-Pipeline. The settings display the GitHub project URL, "GitHub hook trigger for GITScm polling" enabled, Pipeline script from SCM selected with Git as the SCM, repository URL pointing to the flask-backend repo, branch set to */main, and Script Path set to "Jenkinsfile".



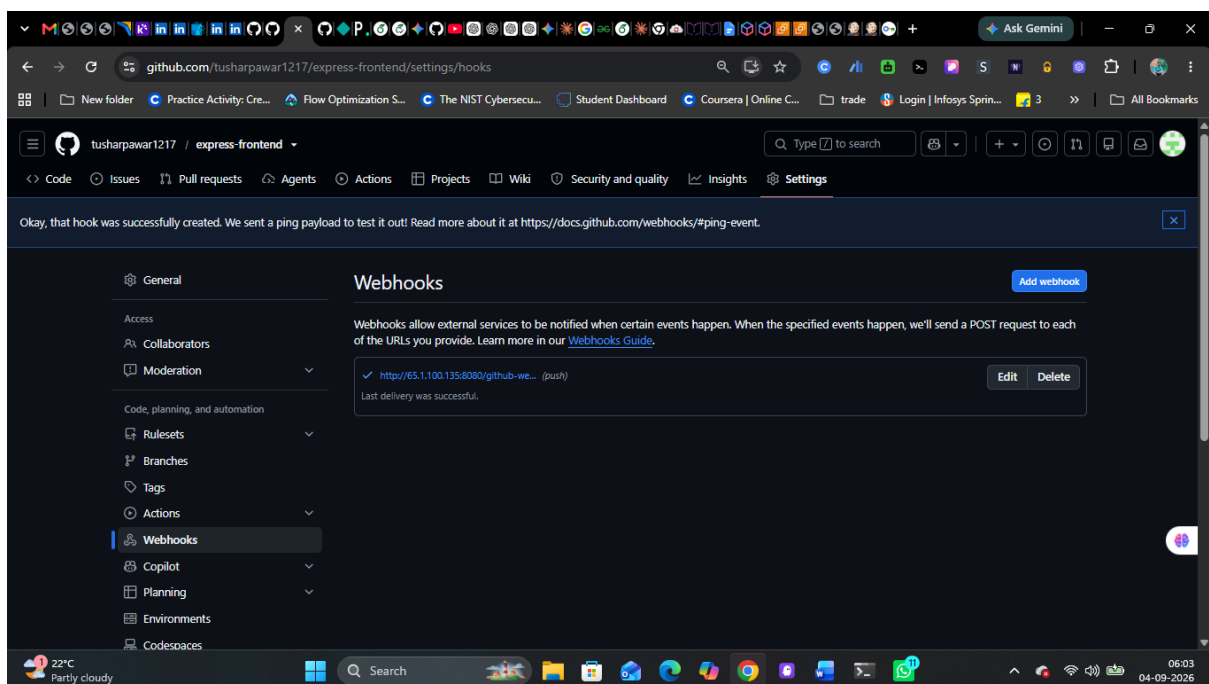
15. This screenshot shows the configuration page for Express-Frontend-Pipeline. Similar to the Flask pipeline, it displays the GitHub project URL for express-frontend, webhook trigger enabled, Git SCM configuration with the express-frontend repository URL, branch */main, and Jenkinsfile path configured correctly.
16. This screenshot displays the Flask-Backend-Pipeline build page showing Build #1 after clicking "Build Now". The page shows build history, build status (blue ball indicating success), build duration, and options to view console output. This confirms manual build execution works correctly.
17. This screenshot displays the Express-Frontend-Pipeline build page showing Build status. The build history shows successful execution (blue indicator), build time, and completion status. This verifies that the Express pipeline can be manually triggered and completes successfully.



18. This screenshot shows the GitHub webhook configuration page for the flask-backend repository. The Payload URL is set to <http://65.1.100.135:8080/github-webhook/>, Content type is application/json, "Just the push event" is selected, and Active is checked. A green checkmark indicates successful webhook configuration.



19. This screenshot displays the GitHub webhook configuration for express-frontend repository. The webhook settings match the Flask configuration with Payload URL pointing to Jenkins, content type as application/json, push event trigger enabled, and active status confirmed with a green checkmark.



20. This screenshot shows Build #2 of Flask-Backend-Pipeline that was automatically triggered by a GitHub webhook after pushing code changes. The build was started by "GitHub push" instead of "user admin", demonstrating successful automatic deployment triggered by git push to the repository.
21. This screenshot displays Build #2 of Express-Frontend-Pipeline automatically triggered via webhook. The build history shows it was initiated by "GitHub push", confirming that the CI/CD automation is working correctly and deployments happen automatically when code is pushed to GitHub.

Jenkins / Flask-Backend-Pipeline

Build View

Delete Pipeline

Configure

Rename

Full Stage View

GitHub

Favorite

Open Blue Ocean

Pipeline Syntax

Builds >

Filter

Today

#7 12:25 AM

Stage View

Average stage times: 572ms

Declarative: Post Actions

#7 Sep 04 05:55 No Changes

#6 Sep 04 05:52 No Changes

#5 Sep 04 05:52 No Changes 572ms

#4 Sep 04 No

22°C Partly cloudy

Search

04-09-2026 06:06

Jenkins / Flask-Backend-Pipeline / #7

Status

Changes

Console Output

Edit Build Information

Open Blue Ocean

Edit build information

Thread Dump

Pause/resume

Replay

Pipeline Steps

Workspaces

Previous Build

Console

Download

Copy

View as plain text

Started by user Tushar Devram Pawar

Obtained Jenkinsfile from git <https://github.com/tusharpawar1217/flask-backend.git>

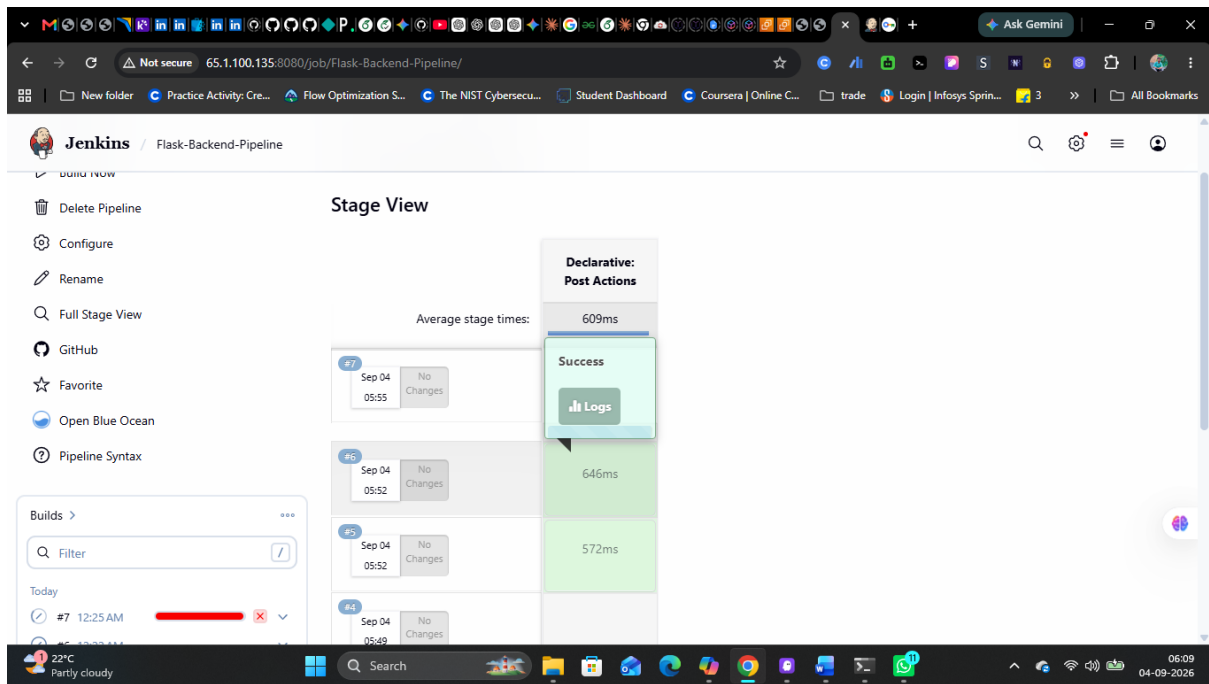
[Pipeline] Start of Pipeline

[Pipeline] node

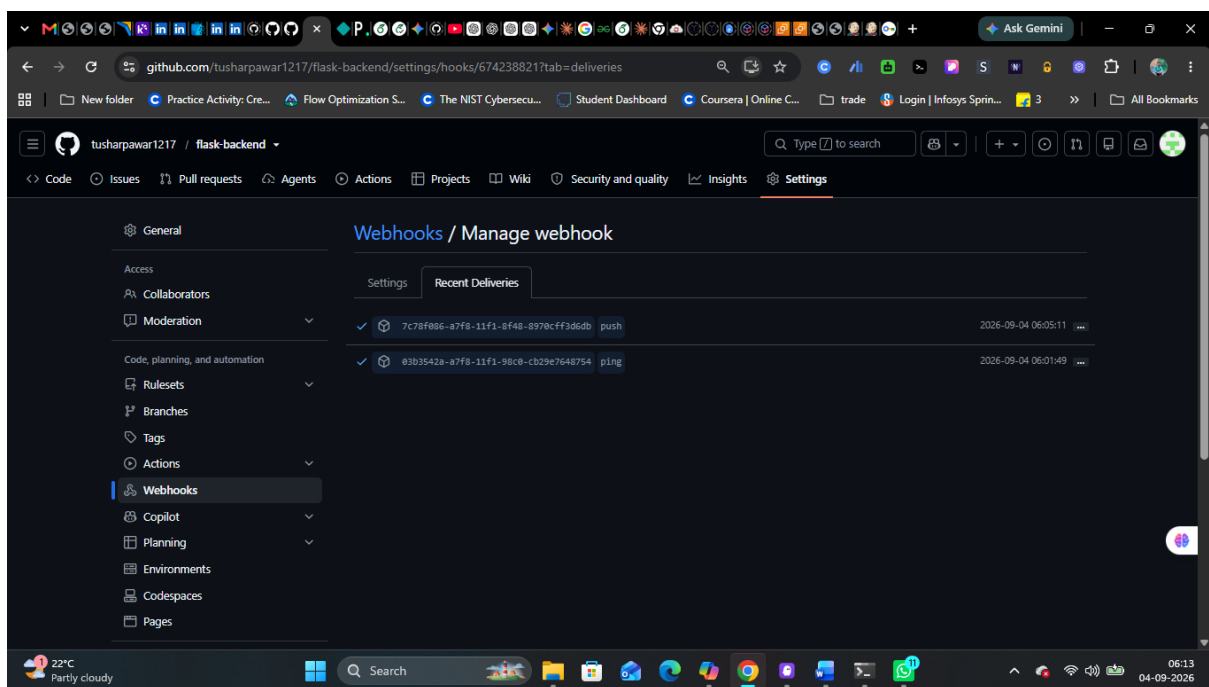
Still waiting to schedule task

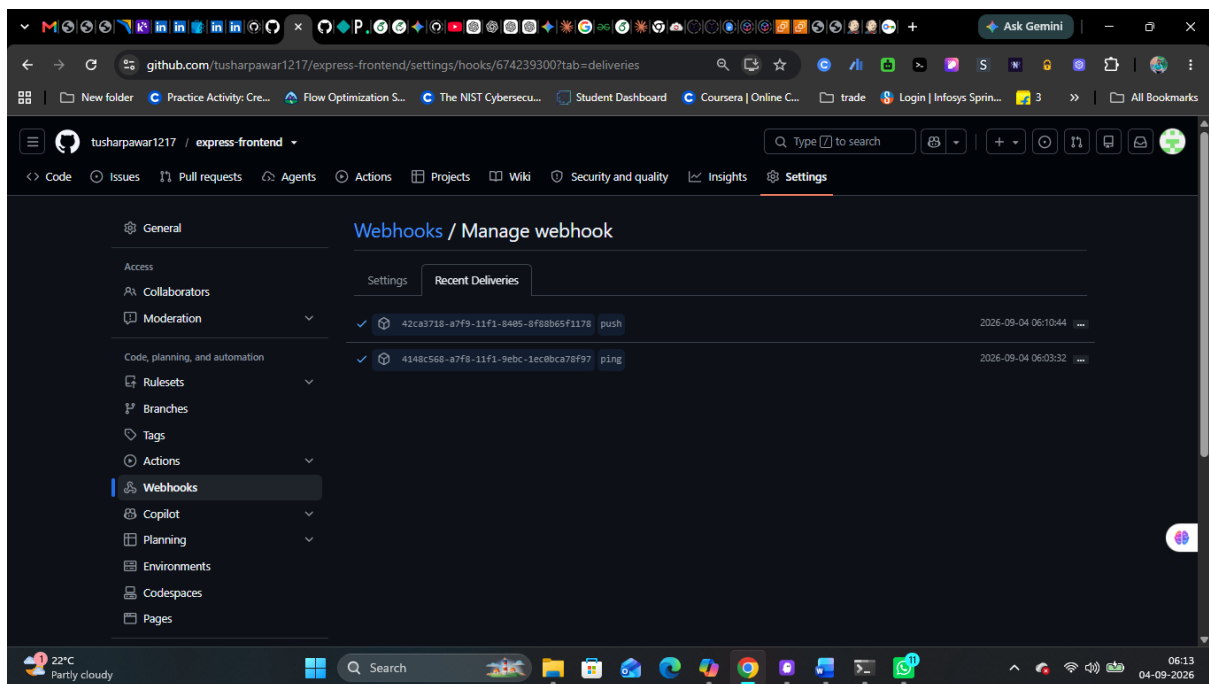
Waiting for next available executor

04-09-2026 06:06

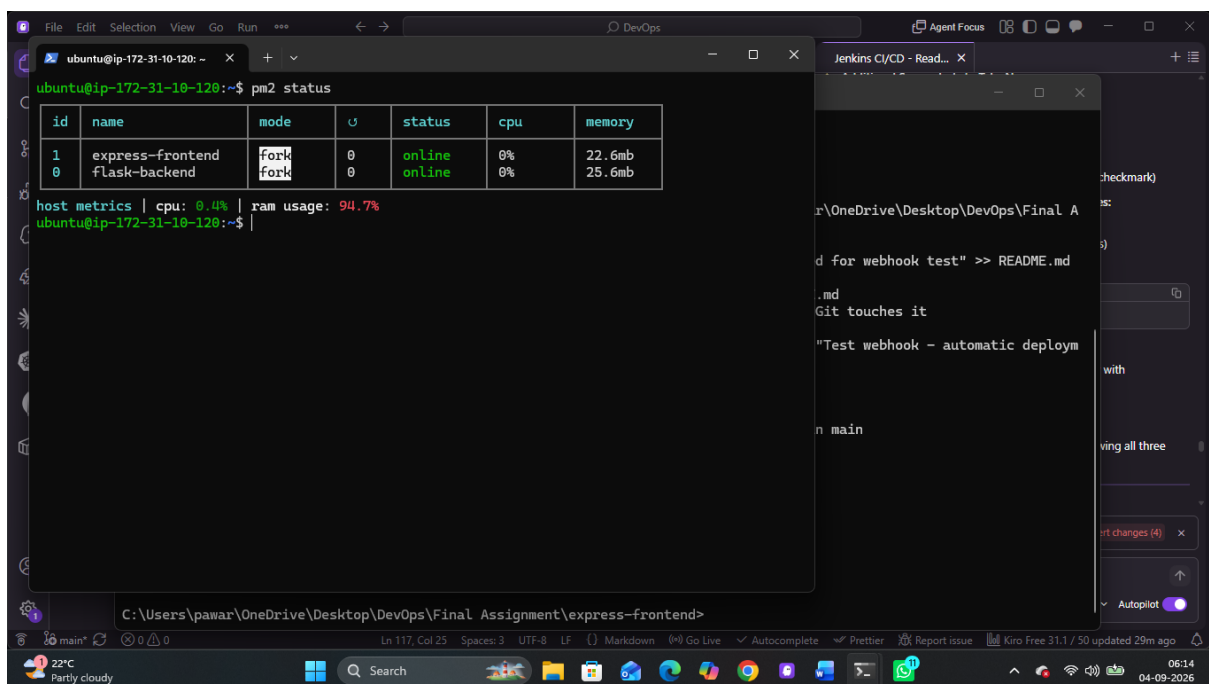


22. This screenshot shows the GitHub webhook "Recent Deliveries" tab displaying successful webhook delivery with HTTP 200 response code and green checkmark. This confirms that GitHub successfully communicated with Jenkins when code was pushed.





23. This screenshot shows the final PM2 status output displaying both flask-backend and express-frontend processes running with "online" status, low CPU usage, normal memory consumption, and uptime indicating stable operation. Both apps show 0 restarts, indicating reliable deployment.



24. This screenshot displays the Jenkins dashboard showing both Flask-Backend-Pipeline and Express-Frontend-Pipeline with multiple successful builds. The weather icons show sunny (100% success rate), and the last success times are recent, demonstrating stable and reliable CI/CD pipelines.

65.1.10

Ask Gemini

Express Frontend

Ask Gemini

Not secure 65.1.100.135:5000

Not secure 65.1.100.135:3000

New folder

Practice Activity: Cre...

Pretty-print

```
{
  "endpoints": {
    "health": "/api/health",
    "item by id": "/api/items/\u003Cid\u003E",
    "items": "/api/items"
  },
  "message": "Flask Backend API",
  "version": "1.0.0"
}
```

DevOps CI/CD Project

Flask Backend + Express Frontend with Jenkins Pipeline

Express Frontend

Online

Flask Backend

Online

Backend URL

http://localhost:5000

Add New Item

Jenkins

Build Queue (1)

part of Flask-Backend-Pipeline #7

Build Executor Status

0/2

Built-In Node

offline

Add description

All

Express-Frontend-Pipeline

Built 37 min ago, took 10 min

Flask-Backend-Pipeline

Built 25 min ago, took 25 min and counting